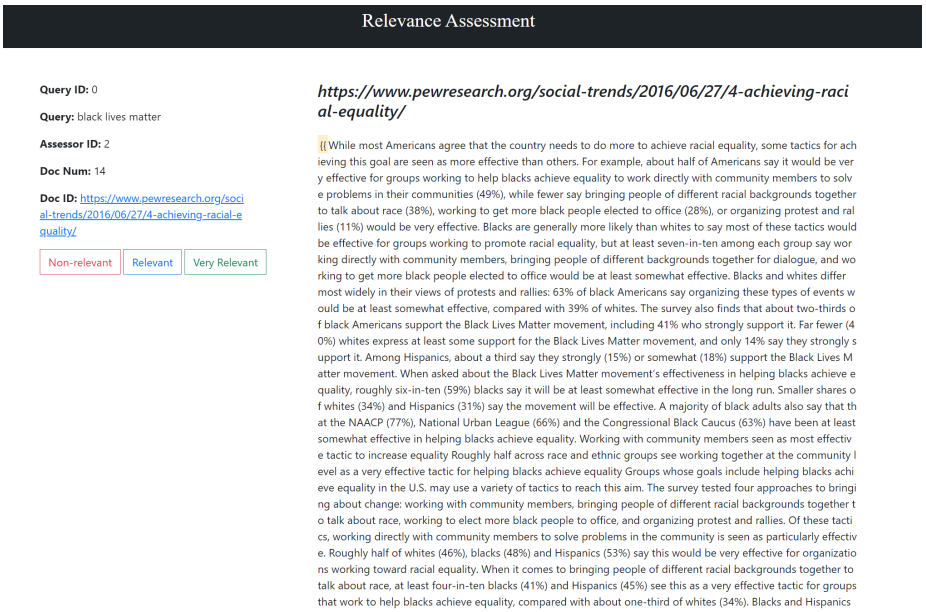
HomeWork-5 Report Template

1. Methodology
    a. Obtaining Queries
        i. Description of the assigned queries
            1. Black lives matter
            2. Women's rights
            3. Racial Justice
            4. Disability Rights
        ii. Explanation of the relevance to your vertical search engine topic
            1. Relevance was judged on whether these documents would show up if I did a Google search on these queries

    b. Assessment Graphical Interface
        i. Features of the interface
            1. Features were seeing the highlighted query, then having the interface to choose between non-relevant, relevant, and very releveant
        ii. Screenshots of the interface



        iii.
    c. Manual Assessments
        i. Approach to assessing the documents
            1. I choose the options on whether if these documents would actual show up in the Google search queries
        ii. Sample assessments with explanations
            1. 2 sunkwan https://en.wikipedia.org/wiki/Racism_in_the_United_States 2
            2. For example, the above query was very relevant to the topic of racial justice since United States does impact a lot on racism

2. Implementation of trec_eval

a. Pseudocode
    i. Qrel = get_qrel_source(qrel_source_path)
    ii. Trec = get_trec_source(trec_source_path)
    iii. For QueryID in sorted(trec.keys()):
        1. For DocID in sorted(qrel):
            a. Compute_precision()
            b. Compute_recall()
            c. Compute_nDCG()
            d. Compute_f1()
        2. Compute_R-precision()
        3. Compute_Average_precision()
    iv. Compute_mean_R-precision()
    v. Copmute_mean_Average_precision()
    vi. Compute_k_documents()
b. Code snippets

```
import math
import sys
from collections import defaultdict

def eval_print(QueryID, num_ret, num_rel, num_rel_ret, avg_prec, cutoffs,
nDCG, prec_at_cutoffs, recall_at_cutoffs, f1_at_cutoffs, r_prec):
   print("\nQueryid (Num):    %5d\n" % QueryID)
   print("Total number of documents over all queries\n")
   print("    Retrieved:    %5d\n" % num_ret)
   print("    Relevant:     %5d\n" % num_rel)
   print("    Rel_ret:      %5d\n" % num_rel_ret)
   print("Average precision (non-interpolated) for all rel docs(averaged over
queries)\n")
   print("                  %.4f\n" % avg_prec)
   print("@k:\n")
   for i in range(len(cutoffs)):
       print("  At    {} docs:   nDCG - {:.4f}, Precision - {:.4f}, Recall -
{:.4f}, F1 - {:.4f}\n".format(cutoffs[i], nDCG[i], prec_at_cutoffs[i],
recall_at_cutoffs[i], f1_at_cutoffs[i]))
   print("R-Precision (precision after R (= num_rel for a query) docs
retrieved):\n")
   print("    Exact:        %.4f\n" % r_prec)

def run():
   print_all_queries = 0
   qrel_file = None
   trec_file = None
   QREL_data = None
   qrel = defaultdict(dict)
   trec = defaultdict(dict)
   num_rel = {}
   tot_num_ret = 0
   tot_num_rel = 0
   tot_num_rel_ret = 0

   cutoffs = [5, 10, 20, 50, 100]
```

```
    sum_prec_at_cutoffs = [0] * len(cutoffs)
    sum_recall_at_cutoffs = [0] * len(cutoffs)
    sum_f1_at_cutoffs = [0] * len(cutoffs)
    sum_nDCG = [0] * len(cutoffs)
    avg_prec_at_cutoffs = [0.0] * len(cutoffs)
    avg_recall_at_cutoffs = [0.0] * len(cutoffs)
    avg_f1_at_cutoffs = [0.0] * len(cutoffs)
    avg_nDCG = [0.0] * len(cutoffs)
    sum_avg_prec = 0
    sum_r_prec = 0

    if len(sys.argv) < 3 or len(sys.argv) > 4:
        print("Usage:  trec_eval [-q] <qrel_file> <trec_file>\n\n")
        return

    if len(sys.argv) == 3:
        qrel_file = sys.argv[1]
        trec_file = sys.argv[2]
    else:
        qrel_file = sys.argv[2]
        trec_file = sys.argv[3]
        print_all_queries = 1

    try:
        with open(qrel_file, 'r') as f:
            QREL_data = f.read().split("\n")
    except:
        print("Could not open qrel file")

    for q in QREL_data:
        if q != "":
            QueryID, AssesorID, DocID, Grade = q.split()
            QueryID = int(QueryID)
            Grade = int(Grade)
            qrel[QueryID].update({DocID : Grade})
            num_rel[QueryID] = num_rel.get(QueryID, 0) + Grade

    try:
        with open(trec_file, 'r') as f:
            TREC_data = f.read().split("\n")
    except:
        print("Could not open trec file")

    for t in TREC_data:
        if t != "":
            QueryID, _, DocID, _, Score, _ = t.split()
            QueryID = int(QueryID)
            Score = float(Score)
            trec[QueryID].update({DocID : Score})

    num_topics = 0
    for QueryID in sorted(trec.keys()):
        if num_rel.get(QueryID) is None:
```

```
            continue

        num_topics += 1
        href = trec[QueryID]
        prec_list = [0.0] * 1001
        rec_list = [0] * 1001
        f1_list = [0] * 1001
        iDCG_list = [1] * 1001
        DCG_list = [0] * 1001
        nDCG_list = [0] * 1001

        for i in range(1, 1001):
            if i < num_rel.get(QueryID):
                iDCG_list[i] = iDCG_list[i - 1] + (1 / (math.log(i + 2, 2)))
            else:
                iDCG_list[i] = iDCG_list[i - 1]

        num_ret = 0
        num_rel_ret = 0
        sum_prec = 0

        for DocID in sorted(href, key = lambda x: (-href[x], x)):
            num_ret += 1
            rel = qrel.get(QueryID, {}).get(DocID, None)
            if rel:
                sum_prec += rel * (1 + num_rel_ret) / num_ret
                num_rel_ret += rel
            prec_list[num_ret] = num_rel_ret / num_ret
            rec_list[num_ret] = num_rel_ret / num_rel.get(QueryID, 0)
            DCG_list[num_ret] = num_rel_ret
            nDCG_list[num_ret] = DCG_list[num_ret] / iDCG_list[num_ret]
            if ((num_rel_ret / num_ret) + (num_rel_ret / num_rel.get(QueryID,
0))) == 0:
                f1_list[num_ret] = 0
            else:
                f1_list[num_ret] = (2 * (num_rel_ret / num_ret) * (num_rel_ret
/ num_rel.get(QueryID, 0))) / ((num_rel_ret / num_ret) + (num_rel_ret /
num_rel.get(QueryID, 0)))
            if num_ret >= 1000:
                break

        avg_prec = sum_prec / num_rel.get(QueryID, 0)

        final_recall = num_rel_ret / num_rel.get(QueryID, 0)

        final_DCG = num_rel_ret

        for i in range(num_ret + 1, 1001):
            prec_list[i] = num_rel_ret / i
            rec_list[i] = final_recall
            DCG_list[i] = final_DCG
            nDCG_list[i] = final_DCG / iDCG_list[i]
```

```
        prec_at_cutoffs = [prec_list[cutoff] for cutoff in cutoffs]
        recall_at_cutoffs = [rec_list[cutoff] for cutoff in cutoffs]
        f1_at_cutoffs = [f1_list[cutoff] for cutoff in cutoffs]
        nDCG = [nDCG_list[cutoff] for cutoff in cutoffs]

        if num_rel.get(QueryID, 0) > num_ret:
            r_prec = num_rel_ret / num_rel.get(QueryID, 0)
        else:
            int_num_rel = int(num_rel.get(QueryID, 0))  # Integer part.
            frac_num_rel = num_rel.get(QueryID, 0) - int_num_rel  # Fractional
part.
            r_prec = (1 - frac_num_rel) * prec_list[int_num_rel] +
frac_num_rel * prec_list[
                int_num_rel + 1] if frac_num_rel > 0 else
prec_list[int_num_rel]

        max_prec = 0
        for i in range(1000, 0, -1):
            if prec_list[i] > max_prec:
                max_prec = prec_list[i]
            else:
                prec_list[i] = max_prec

        if print_all_queries == 1:
            eval_print(QueryID, num_ret, num_rel.get(QueryID, 0), num_rel_ret,
avg_prec, cutoffs, nDCG, prec_at_cutoffs, recall_at_cutoffs, f1_at_cutoffs,
r_prec)

        tot_num_ret += num_ret
        tot_num_rel += num_rel.get(QueryID, 0)
        tot_num_rel_ret += num_rel_ret

        for i in range(len(cutoffs)):
            sum_prec_at_cutoffs[i] += prec_at_cutoffs[i]
            sum_recall_at_cutoffs[i] += recall_at_cutoffs[i]
            sum_f1_at_cutoffs[i] += f1_at_cutoffs[i]
            sum_nDCG[i] += nDCG[i]

        sum_avg_prec += avg_prec
        sum_r_prec += r_prec

    print("Error due to %d\n" % num_topics)
    for i in range(len(cutoffs)):
        avg_prec_at_cutoffs[i] = sum_prec_at_cutoffs[i] / num_topics
        avg_recall_at_cutoffs[i] = sum_recall_at_cutoffs[i] / num_topics
        avg_f1_at_cutoffs[i] = sum_f1_at_cutoffs[i] / num_topics
        avg_nDCG[i] = sum_nDCG[i] / num_topics

    mean_avg_prec = sum_avg_prec / num_topics
    avg_r_prec = sum_r_prec / num_topics

    eval_print(num_topics, tot_num_ret, tot_num_rel, tot_num_rel_ret,
```

```
            mean_avg_prec, cutoffs, avg_nDCG, avg_prec_at_cutoffs,
avg_recall_at_cutoffs, avg_f1_at_cutoffs, avg_r_prec)

run()

# eval_print(0.250, 25000, 0, 0, 0, 0, 0, 0)
```

3. Evaluation Results
    a. Results of Manual Assessments
        i. Compilation of assessments in QREL format

```
1. 0 sunkwan
   https://commons.wikimedia.org/wiki/Category:Black_Lives_Matter 0
2. 0 sunkwan
   https://en.wikipedia.org/wiki/List_of_Black_Lives_Matter_street_murals 1
3. 0 sunkwan
   https://en.wikipedia.org/wiki/BlackLivesMatter 2
4. 0 sunkwan
   https://en.wikipedia.org/wiki/Black_Lives_Matter 2
5. 0 sunkwan
   https://en.wikipedia.org/wiki/All_Lives_Matter 2
6. 0 sunkwan
   https://en.wikipedia.org/wiki/Black_Lives_Matter_Plaza 1
7. 0 sunkwan
   https://en.wikipedia.org/wiki/George_Floyd_protests_in_the_United_Kingdom 0
8. 0 sunkwan
   https://en.wikipedia.org/wiki/Black_Lives_Matter_Global_Network_Foundation 2
9. 0 sunkwan
   https://en.wikipedia.org/wiki/Black_Lives_Matter_movement_in_popular_culture 2
10. 0 sunkwan
    https://en.wikipedia.org/wiki/George_Floyd_protests_in_the_Netherlands 0
11.
```

        ii. Discussion on the assessment results
            1. Assessments were taken into Average to see if the document was relevant or not. If on average, the documents were above 1, it was relevant.

    b. trec_eval Results
        i. Presentation of results from your trec_eval

Error due to 25

Queryid (Num): 25

Total number of documents over all queries

Retrieved: 25000

Relevant: 1832

Rel_ret: 1331

Average precision (non-interpolated) for all rel docs(averaged over queries)

0.3059

@k:

At 5 docs: nDCG - 0.7868, Precision - 0.5200, Recall - 0.0713, F1 - 0.1110

At 10 docs: nDCG - 0.9968, Precision - 0.4760, Recall - 0.1166, F1 - 0.1572

At 20 docs: nDCG - 1.2054, Precision - 0.4180, Recall - 0.1855, F1 - 0.2084

At 50 docs: nDCG - 1.5270, Precision - 0.3416, Recall - 0.3117, F1 - 0.2634

At 100 docs: nDCG - 1.7696, Precision - 0.2556, Recall - 0.4639, F1 - 0.2614

R-Precision (precision after R (= num_rel for a query) docs retrieved):

Exact: 0.3350

ii. Comparison with provided trec-eval (if applicable)
Error due to 25

Queryid (Num): 25
Total number of documents over all queries
Retrieved: 25000
Relevant: 1832
Rel_ret: 1331
Interpolated Recall - Precision Averages:
at 0.00 0.7610
at 0.10 0.6070
at 0.20 0.4722
at 0.30 0.4142
at 0.40 0.3586
at 0.50 0.2935
at 0.60 0.2284
at 0.70 0.1857

at 0.80 0.1187
at 0.90 0.0673
at 1.00 0.0095
Average precision (non-interpolated) for all rel docs(averaged over queries)
0.3059
Precision:
At 5 docs: 0.5200
At 10 docs: 0.4760
At 15 docs: 0.4293
At 20 docs: 0.4180
At 30 docs: 0.3853
At 100 docs: 0.2556
At 200 docs: 0.1762
At 500 docs: 0.0952
At 1000 docs: 0.0532
R-Precision (precision after R (= num_rel for a query) docs retrieved):
Exact: 0.3350

iii. Analysis of the results
- The results were similar to the original trec_eval.pl scores.

6. Extra Credit (If Attempted)
- Description of the extra credit task(s) undertaken
  - GUI Web Interface + Highlighted query terms